

Front End Style Guide

Contents

1. [Naming](#)
 - HTML
 - CSS
 - Javascript
2. [Formatting](#)
 - HTML
 - Javascript

Naming

Just like in your Apex code:

Appropriate naming helps us glean useful information quickly. Conventions facilitate self-documenting code that improves readability and consistency.

HTML

All tag names and attributes should be all lowercase and class and Id names should be **kebab-case**

```
<button type="button" id="delete-button" class="xl red button"
role="menu">
    Delete
</button>
```

Javascript

Files

When naming files follow these rules:

- No spaces in file names
- Files should be named in all lower case
- File names with multiple words should be named with **kebab-case**
- Use periods at the end of file names to denote specific versions of code, or to denote what type of file it is:
 - **date-picker.js**
 - **date-picker.min.js**
 - **contact.ctrl.js**

Variables

When naming variables, follow the same rules as the Apex style guide, make sure the name describes what the variable is for, and use **camelCase**

Constants

When you need a constant, make sure to use the `const` keyword, instead of `var` or `let`, and name it `ALL_CAPS_WITH_UNDERSCORES`

Functions

When naming functions, make sure to follow the same rules as the Apex style guide, remembering to make the function name describe what it is doing

- use `camelCase`
- if the function returns a value, "get" should be prepended to the method name
- if the function sets a value, "set" should be prepended to the method name
- if the function returns a boolean, make sure to phrase the name as a question, e.g. `isEmpty()`, `hasChildren()`, `canConnect()`

[Back to Contents](#)

Formatting

HTML

The very first thing in your html file (even before the html tag) should be the doctype (salesforce adds this for you). The doctype is important because it tells the browser what XHTML standards you will be using. At minimum, your page should have this:

```
<!doctype html>
```

After that should come your html tag, followed by a head and a body:

```
<!doctype html>
<html lang="en-US">
  <head>
  </head>
  <body>
  </body>
</html>
```

Your head should at minimum have a title, and the viewport meta tags

```
<!doctype html>
<html lang="en-US">
  <head>
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Make sure this is meaningful because it will
display on the browser</title>
```

```
        </head>
        <body>
        </body>
</html>
```

Stylesheet imports go in the head and script imports are the last thing at the bottom of the page. The reason for this is that stylesheets are required for your markup to look nice, scripts are not. Scripts add functionality, so put them at the bottom so they get downloaded last so there isn't just a big old blank screen for a second while the browser loads all the files it needs.

HTML attributes should not have any space between the attribute name, the =, and the opening ". Remember to close all your elements (including empty ones like the input below):

```
<form>
    <input type="text" />
    <p>Some paragraph</p>
</form>

```html
// GOOD
<button type="button">
 Click Me
</button>

// BAD
<button type = "button">
 Click Me
</button>
```

Do your best to avoid making a line more than 80 characters, if you start to get that long, make a new line.

```
<button class="slds-button slds-button_icon slds-button_icon-border-filled
slds-button_icon-x-small slds-shrink-none"
 aria-haspopup="true">
 ...
</button>

<p>
 This paragraph is going to have some really inspiring stuff in it
 and I really hope everybody takes the time to read all of it.
 There is really no reason to make this all in one line, the width
 of your markup in the browser is handled by styling tags
 and not by the line number it is in your file.
</p>
```

## Javascript

## General

Just like our Apex style guide, use tabs instead of spaces for indents.

All if statements should have an opening curly bracket at the end of the starting line, and the closing one on a new line (even if the block is only 1 line long).

```
if(arr.isEmpty()) {
 return ERROR_MESSAGE;
}
```

Spaces need to go between all operators except for unary operators:

```
var num = x + y;
++i;
```

## Functions

(I might take this out b/c I think that this messes w/ IE, need to verify) Anonymous functions are one of the things that makes Javascript great, but you should still name them.

- Naming your anonymous function helps with debugging, promotes writing reusable code, and eliminates ambiguities so other developers do not need to read the entire anonymous function body to learn what it does

```
// AVOID
setTimeout(function() {
 alert('Hello!');
}, 1000);

// GOOD
setTimeout(function sayHello() {
 alert('Hello!');
}, 1000);
```

**Always avoid putting things on the global scope, scope your functions to an IIFE** When we do not scope our functions and variables, they automatically go to the global scope where everything can access them.

Instead, use the Revealing Module Pattern to encapsulate your code:

```
window.revealedModule = (function() {
 'use strict';

 var privateVar = "I am Gob",
 publicVar = "Hello Gob!";
```

```
 init();

 return {
 setName: publicSetName,
 greeting: publicVar,
 getName: publicGetName
 };

 function init() {
 // any initialization logic you might need to do
 }

 // Public methods
 function publicSetName(name) {
 privateVar = name;
 }

 function publicGetName() {
 privateFunction();
 }

 // Private methods
 function privateFunction() {
 console.log("Name: " + privateVar);
 }
})();
```

## Objects

- Object properties should be split up so that each is on its own line
  - The same applies for nested arrays and objects inside object properties
- Use a dangling comma on the last property of the object
- Avoid using quotes on object properties. If you must use them, use them for all object properties

```
var book = {
 priceUsd: 29.99,
 isbn: '013235022',
 tags: [
 'books',
 'programming',
 'computers',
 'software development'
],
 publisher: 'Prentice',
 author: 'Robert C. Martin',
 title: 'Clean Code',
}
```